

Pipeline de Datos (ETL) - Prueba Técnica

Este proyecto implementa un pipeline robusto y escalable de Extracción, Transformación y Carga (ETL) desarrollado en Python y PostgreSQL para procesar y consolidar información de almacenes, artículos, movimientos y ventas.

Arquitectura y Decisiones de Diseño

Durante el desarrollo de esta solución se tomaron decisiones clave para asegurar que el pipeline sea eficiente, robusto y de grado de producción:

1. Desacoplamiento de Ingesta y Consolidación (Patrón Medallion)

- **Enfoque:** Se implementó una arquitectura ELT utilizando el patrón Medallion.
- **Capa Bronze (Staging):** Python extrae la data en bloques (chunks), aplica transformaciones ligeras (*snake_case*, limpieza de espacios y descarte de nulos) mediante Pandas, y deposita la información en PostgreSQL (tablas *stg_**).
- **Capa Silver (Producción):** Mediante el script *silver_transform.sql*, se eliminan los registros duplicados a nivel global y se garantiza la integridad referencial antes de consolidar la información en los catálogos y tablas de hechos finales.

2. Procesamiento Eficiente por Bloques (Chunking)

- **Enfoque:** La lectura de archivos CSV se realiza por bloques (*chunksize=500* o configurable) usando generadores de Pandas.
- **Razón:** Garantiza que el uso de memoria RAM del pipeline se mantenga constante e independiente del tamaño del archivo de entrada, evitando fallas por memoria insuficiente (*OutOfMemory*) al procesar millones de registros.

3. Validación de Calidad e Integridad de Datos Basada en Esquemas

- **Enfoque:** Toda la estructura de datos se valida declarativamente mediante esquemas en formato JSON (*schemas/*.json*).
- **Acciones:**
 - **Estandarización:** Renombrado dinámico de columnas de nombres "raw" (originales en el archivo) a nombres "clean" (estandarizados en BD).
 - **Limpieza de Texto:** Remoción de espacios en blanco sobrantes (*strip()*) en todas las columnas de tipo texto.
 - **Filtrado de Nulos:** Identificación automática de registros inválidos que contienen nulos en columnas obligatorias (*required: true*) y descarte preventivo con registro en los logs para auditoría.

4. Ciclo de Vida de Tablas Staging

- **Enfoque:** Las tablas de staging son truncadas **una vez al inicio de todo el pipeline**.

- **Resultado:** Los bloques individuales correspondientes a la misma tabla se insertan secuencialmente (append) acumulándose en staging de forma limpia y garantizando que no se dupliquen registros de ejecuciones previas incompletas.

5. Reprocesamiento Seguro (Idempotencia)

- **Enfoque:** Para garantizar que el proceso pueda ejecutarse múltiples veces sin generar datos duplicados, se implementaron dos estrategias en la capa Silver:
- Los catálogos (**almacenes**, **articulos**) se actualizan mediante **UPSERT** (**ON CONFLICT DO UPDATE**).
- Las tablas de hechos (**movimientos**, **ventas**) utilizan el patrón **DELETE-INSERT**, borrando dinámicamente los días a reprocesar antes de insertar la nueva carga.

Hallazgo de Calidad de Datos (Manejo de Duplicados y Negativos): Durante el desarrollo, se comprobó que el archivo **movimientos.csv** contenía registros duplicados idénticos. Al ser eliminados por la Capa Silver (para garantizar la unicidad e integridad), el volumen real de entradas disminuyó, ocasionando que las ventas superaran a las recepciones y generando inventarios matemáticamente negativos. Para efectos prácticos y de demostración visual del ejercicio, se optó por modificar manualmente las fechas de estas entradas duplicadas en el archivo CSV (distribuyéndolas en meses posteriores) para simular un reabastecimiento continuo y mantener los saldos de inventario en números positivos.

6. Particionamiento de Datos

- **Enfoque:** Al ser este un ejercicio de prueba, se optó por usar tablas estándar para agilizar el desarrollo y las pruebas locales.
- **Consideración Productiva:** En un entorno real con gran volumen de datos, las tablas de hechos deberían estar particionadas (por ejemplo, por mes). La estrategia exacta dependerá de la versión del motor de base de datos definitivo.

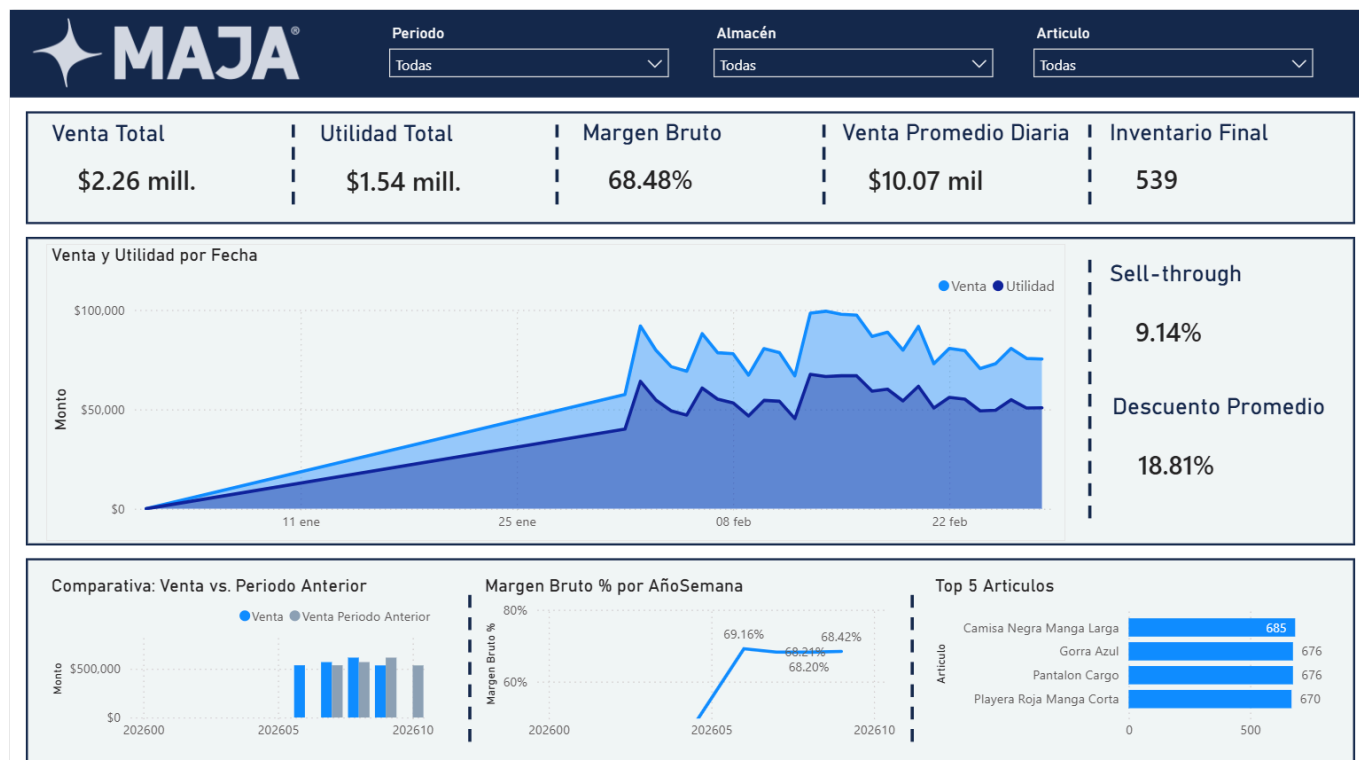
7. Seguridad y Configuración

- **Enfoque:** Las credenciales de conexión fueron desacopladas del código fuente para evitar *hardcoding*. Se requiere un archivo local **.env** en la raíz del proyecto para que el pipeline lea la configuración dinámicamente mediante **python-dotenv**.

Para efectos prácticos se deja el archivo **.env** en el proyecto para que pueda ser usado directamente sin necesidad de crearlo.

8. Visualización y Modelado en Power BI

La capa de visualización se construyó conectando Power BI directamente a la base de datos PostgreSQL, implementando un modelo técnico optimizado:



- **Conectividad:** Conexión directa a PostgreSQL mediante el conector nativo para consumir los datos de la capa Gold.
- **Optimización del Modelo (Fact/Dimension):**
 - Se realizó un proceso de combinación de tablas en Power Query (`articulos`, `gold_dim_articulo_costo` y `gold_dim_articulo_precio`) para consolidar la información en una sola tabla maestra.
 - Se deshabilitó la carga de las tablas fuente originales (`articulos`, `costos`, `precios`) para evitar redundancia y optimizar el uso de memoria.
- **Arquitectura de Datos:** Se estructuró el modelo bajo un esquema tipo estrella, relacionando la tabla de hechos (`gold_indicadores_negocio`) con tablas dimensionales.
- **Calendario e Inteligencia de Tiempo:** Se integró una tabla de calendario mediante `CALENDARAUTO()` y se aplicó un filtrado dinámico en los objetos visuales para mostrar exclusivamente las semanas con actividad comercial (`movimientos > 0`), evitando el ruido de periodos sin transacciones.
- **Lógica de KPIs y Restricciones Técnicas:**
 - **Cálculo de Inventario:** Se configuró la medida para que el Inventario Final se calcule basándose estrictamente en el valor reportado en la última fecha seleccionada en los filtros, asegurando precisión en el saldo de stock al momento del análisis.
 - **Reemplazo de Ticket Promedio:** Ante la ausencia de un identificador de transacción único (`id_transaccion`) en la fuente agregada, se descartó el cálculo del ticket promedio para evitar sesgos. En su lugar, se implementó el KPI "`Venta Promedio Diaria`", proporcionando una métrica consistente con la granularidad de los datos.

Supuestos Realizados

Durante el análisis de los datos fuente y el desarrollo del pipeline, se identificaron y asumieron los siguientes puntos:

Sobre el Modelo de Datos

1. **Costo Único por Artículo:** Se asume que cada artículo tiene un único costo de adquisición vigente. El catálogo `gold_dim_articulo_costo` toma el costo del movimiento de entrada más reciente por artículo como el costo representativo para valuar el inventario y las ventas.
2. **Precio Único por Artículo:** De forma análoga al costo, se asume un único precio de lista por artículo. El catálogo `gold_dim_articuloPrecio` toma el precio más reciente registrado en las ventas.
3. **Precio en Ventas como Precio Real Cobrado:** El campo `precio_total` de la tabla de ventas se interpreta como el monto **efectivamente cobrado** al cliente (ya incluyendo cualquier descuento aplicado en el punto de venta). La diferencia entre $(precio_lista * unidades)$ y `precio_total` se calcula como el **descuento otorgado**.
4. **Identificadores como Claves Enteras:** Los campos `almacen_id` y `articulo_id` se identificaron como identificadores numéricos (a pesar de estar almacenados como cadenas en los CSV originales), por lo que se tipificaron como `BIGINT` en la base de datos para optimizar los JOINS.
5. **Tabla de Ventas Independiente del Inventario:** Se asume que el sistema fuente registra ventas y movimientos de almacén de forma independiente. Por ello, se utiliza un `FULL OUTER JOIN` en la capa de indicadores para garantizar que días con movimientos pero sin ventas, o viceversa, no se pierdan del reporte.

Sobre la Calidad de los Datos

6. **Duplicados en CSV de Movimientos:** Se detectaron registros duplicados exactos en el archivo `movimientos.csv`, particularmente en entradas del `2026-01-01`. La Capa Silver aplica deduplicación automática, pero esto reducía el saldo de inventario causando valores negativos. Como solución práctica para el ejercicio, se redistribuyeron manualmente las fechas de las entradas duplicadas a meses posteriores para mantener saldos positivos y demostrar el flujo completo del pipeline.
7. **Inventario Inicial en Cero:** Se asume que al inicio del periodo analizado (`2026-01-01`) el inventario es **cero para todos los artículos**. El inventario se construye acumulando entradas y salidas desde el primer movimiento registrado mediante funciones de ventana.
8. **Sell-through Basado en Disponibilidad Diaria:** El KPI de Sell-through se calcula como $Ventas / (Inventario\ Inicio + Entradas\ del\ Día)$. Si el inventario disponible de un día es cero, el indicador se reporta como `0` para evitar divisiones por cero.

Sobre la Arquitectura

9. **Entorno de Prueba sin Particionamiento:** Para simplificar el desarrollo local, se utilizaron tablas estándar en la Capa Silver (sin particionamiento por fecha). En un entorno de producción con volúmenes reales, se recomienda implementar particionamiento por fecha.
10. **Recreación Completa de Tablas Gold:** Las funciones de la Capa Gold aplican un patrón `DROP IF EXISTS + CREATE TABLE AS`, lo cual garantiza la idempotencia total en un entorno de prueba. En

producción se debería implementar una estrategia de carga incremental.

Sobre el Modelado en Power BI

11. **Tratamiento de fechas (Calendario):** Se estableció `CALENDARAUTO()` como la tabla central del modelo. Toda relación de tiempo entre hechos y dimensiones depende exclusivamente de esta tabla, evitando el uso de campos de fecha directos de la tabla de hechos para asegurar consistencia.
12. **Periodos Semanales:** Debido a la naturaleza y volumen de la información proporcionada, se asume el uso de periodos semanales como estándar de análisis para las comparativas temporales y la visualización de tendencias, optimizando así la legibilidad de la información.

Estructura del Proyecto

```
├── dags/                # Orquestación con Airflow (DAGs)
├── data/                # Archivos CSV fuente
├── notebooks/          # Análisis exploratorio y pruebas rápidas
├── schemas/            # Esquemas JSON de validación por tabla
├── sql/                # DDL de tablas y procedimientos almacenados de BD
├── src/                # Código fuente modular
│   ├── config.py       # Configuración de variables de entorno y base de datos
│   ├── extract.py      # Módulo de extracción de archivos CSV
│   ├── load.py         # Módulo para carga a tablas PostgreSQL
│   ├── transform.py    # Módulo para estandarización y calidad de datos
│   └── utils.py        # Funciones de logging y utilidades
├── tests/              # Pruebas unitarias
├── docker-compose.yml  # Orquestación de infraestructura de servicios
├── main.py             # Ejecutor alternativo manual en Python
└── README.md           # Documentación del proyecto
```

Requisitos Previos

- **Docker y Docker Compose** instalados (Recomendado para evaluación completa).
- *(Opcional)* Python 3.8+ si se desea ejecutar el pipeline localmente sin Docker.

Configuración y Ejecución (Vía Docker + Airflow)

El proyecto está dockerizado para facilitar su evaluación. Toda la infraestructura (Bases de datos + Orquestador) se levanta con un solo comando.

1. **Levantar la Infraestructura:** Abre una terminal en la raíz del proyecto y ejecuta:

```
docker compose up -d
```

Esto descargará las imágenes e iniciará PostgreSQL (16), la base interna de Airflow (15) y los servicios del orquestador.

2. **Acceder a Apache Airflow:** Espera ~30 segundos para que los servicios inicialicen. Luego abre tu navegador en:

- **URL:** `http://localhost:8085`
- **Usuario:** `admin`
- **Contraseña:** `admin`

3. **Ejecutar el Pipeline:** En la interfaz web de Airflow, busca el DAG llamado `dag_etl_pipeline`.

- Puedes hacer clic en el DAG y ver el "Graph" para observar cómo se ejecutan secuencialmente las capas Bronze, Silver y Gold.

4. **Consultar los Datos Finales:**

- **Host:** `localhost`
- **Puerto:** `5454`
- **Usuario:** `etl_user`
- **Contraseña:** `etl_password`
- **Base de datos:** `etl_db`

La tabla lista para ser conectada al Dashboard es `gold_indicadores_negocio`.

Ejecución Manual (Alternativa sin Docker)

Si cuentas con una base de datos PostgreSQL local corriendo en el puerto `5432`, puedes configurar tus credenciales en el archivo `.env` y ejecutar el orquestador manual de Python:

```
pip install -r requirements.txt
python main.py
```